

LESSON 5: SECURE CODING & PRIVACY BY DESIGN (BUILDING IT IN, NOT ADDING IT ON)

LEGACY: POST-FACTO FIXER
(RUNNING AT 10% CAPACITY)

RISK TREATMENT
MEASURE PRODUCER

AVOID
INJECTION:
It's not a
vaccine,
it's a code
vulnerability!

ISO SECURE CODING
GUIDELINES

INPUT

NEN-7510-2024
PRIVACY PRINCIPLES

Data Protection
Hammer



Smashing bug
for code

PRIVACY-BY-DESIGN CHECKLIST (NEN 7510 CONTROLS)

- ☐ Data Minimization
- ☐ Purpose Limitation
- ☐ Right to be Forgotten Flow
- ☐ Access Control Design
- ☐ OpenMRS Consent Management Integration



CODE SECURING &
PRIVACY-BY-DESIGN
COMPOSER & FORGE

```
<code>  
<developer input>  
<code>  
<code>code name">  
</code>  
<code>  
</code>
```

DEVELOPER

SECURITY-HARDENED
HIS ACCESS
CONTROL

CERTIFIED

PRIVACY-
PRESERVING EHR
ENCRYPTION

CONSENT-BASED
DATA SHARING
LOGIC

PRIVACY-CHECKED

OpenMRS
INTEGRATION
CHECKED

OpenMRS
Integration Check



DESIGN-INCLUDED
BUG SECURITY

SECURE CODING & PRIVACY BY DESIGN

WAAR STAAN WE

Workshop	Thema	Deliverable
WS01	Wet- & regelgeving, NEN-7510, audit mindset	Begrippenkader
WS02	Hardening CI/CD pipeline (CodeQL, Dependabot, SBOM)	Beveiligde pipeline
WS03	Healthcare Risk Assessment (risicomatrix, bow-tie)	Risicorapport
WS04	Compliance Scanning (CVE/CVSS, Snyk, security backlog)	Scanrapport + backlog
WS05	Secure Coding & Privacy by Design	Logging + tests
WS06	Formeel auditrapport	Definitief auditrapport

LEERDOELEN

Na deze workshop kun je...

- ✓ Security-activiteiten koppelen aan elke SDLC-fase en de meerkosten van een laat gevonden bug uitleggen ('shift left'-principe).
- ✓ De OWASP Top 10 toepassen op Java/OpenMRS en kwetsbare patronen herkennen en refactoren naar veilige code.
- ✓ Privacy by Design toepassen
- ✓ Compliant audit-logging implementeren en testen
- ✓ een DPIA-check uitvoeren voor de verwerking van bijzondere persoonsgegevens en de spanning tussen logging en privacy uitleggen.

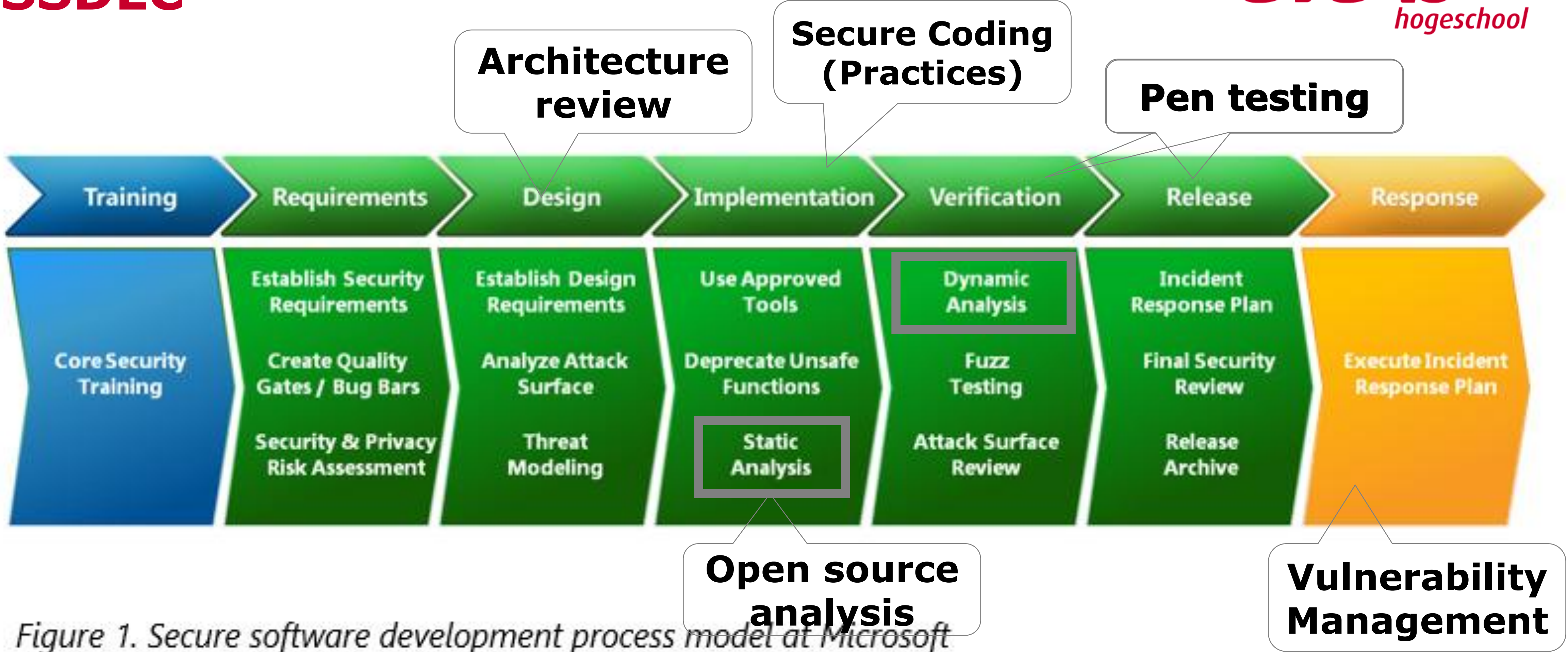
Secure SDLC

- **Security is 'discrete' stap maar een doorlopend proces.**
- Shift Left
 - hoe eerder een probleem gevonden, hoe goedkoper het is om te fixen.

NEN-7510:2024-2 – 8.25 Beveiligen tijdens de ontwikkelcyclus
Beheersmaatregelen voor beveiliging moeten worden geïmplementeerd in alle fasen van de software-ontwikkelcyclus.

Fase	Kosten (relatief)
Requirements	1×
Design	5×
Implementation	10×
Testing	15×
Release/prd	30×

SSDLC



SSDLC | Activiteiten per fase

Fase	Beveiligingsactiviteit	Verantwoordelijke	NEN-7510
Requirements	Beveiligings- & privacyvereisten opstellen; DPIA-check	PO / Security Champion*	5.34, 8.25
Design	Threat modeling (STRIDE/MAPGOOD); attack surface mapping; secure defaults bepalen	Architect / Team	8.25, 8.11
Implementation	Secure coding guidelines volgen; code reviews; hardcoded secrets voorkomen	Developer	8.28
Testing	SAST (CodeQL), DAST (ZAP), unit tests voor security-relevante code, penetratietest	QA / Developer	8.29
Release	SBOM genereren; dependency check; sign & verify artefacten	DevOps / Release mgr	8.25
Operate	Monitoren, patchen, auditlogs analyseren, incident response	Ops / CISO	8.15, 8.8

***Security Champion:** iemand die 'extra' alert is op dit onderwerp

Bow-tie / threat model → Code

Voorbeeld: OpenMRS patientmodule

Dreiging (WS03)	Implementatie
Ongeautoriseerde toegang tot patiëntdata	Fine-grained <code>@PreAuthorize("hasRole('ROLE_CLINICIAN')")</code> op elke endpoint
SQL-injectie via zoekformulier	Hibernate parameterized queries; nooit <code>String.format()</code> in JPQL
Gevoelige data in logbestanden	Logger mag nooit BSN of wachtwoord loggen
Session hijacking	Secure + HttpOnly cookies; session invalidation na uitloggen
Onveilige API-respons	Minimale velden teruggeven (dataminimalisatie in JSON-respons)

Gebruik het threat model als referentie voor de code reviews.

NEN-7510 8.25: *De resultaten van threat modeling moeten worden gebruikt als input voor beveiligingsvereisten en beveiligingsontwerp.*

Attack Surface Analysis

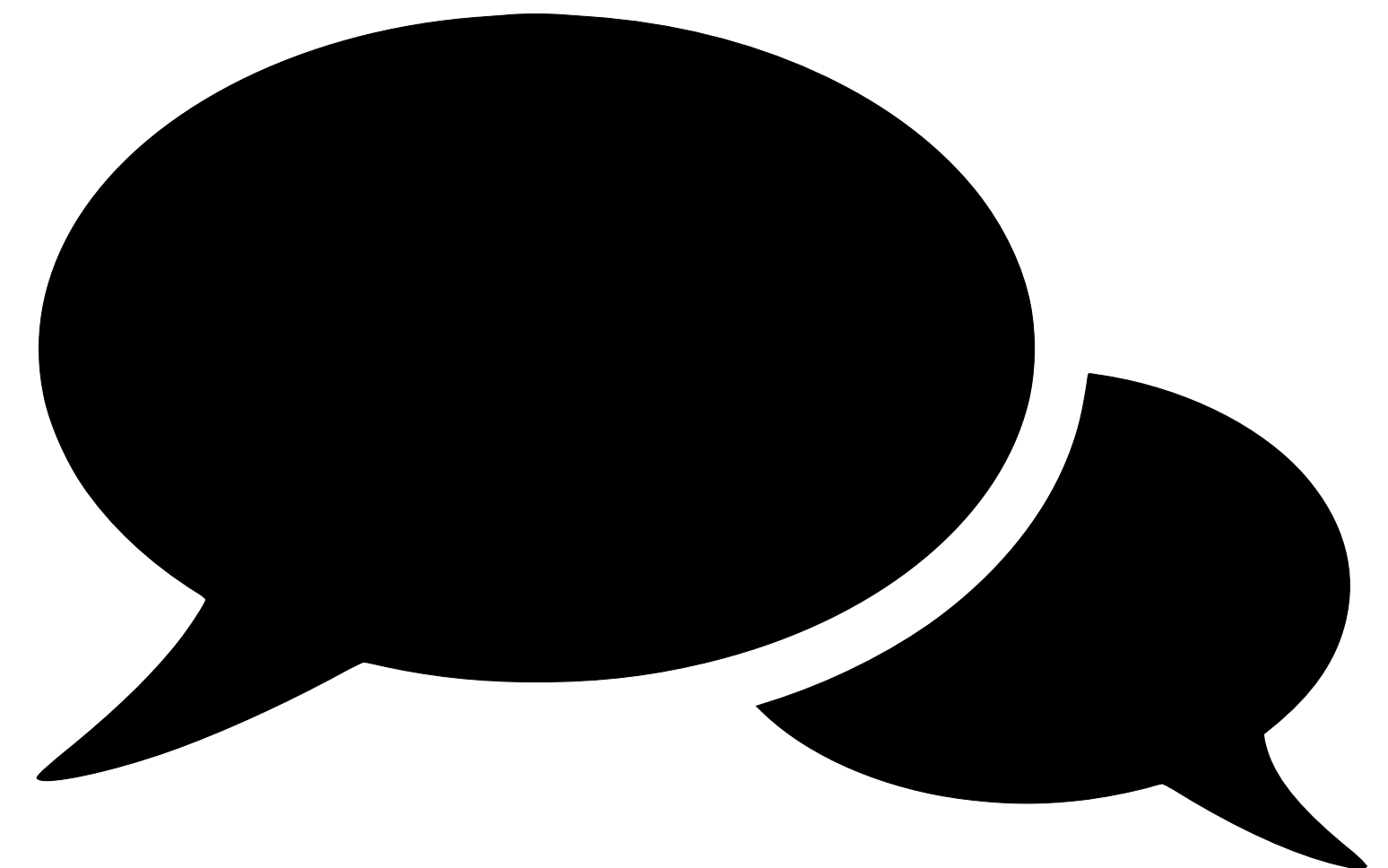
- Attack Surface?
 - Alle punten waarop een aanvaller input kan leveren aan, of output kan onttrekken aan het systeem.

Categorie	Voorbeelden in OpenMRS
Netwerk-endpoints	HTTP-poorten, SSH, database-poort
API-endpoints	REST <code>/ws/rest/v1/</code> , FHIR2 <code>/ws/fhir2/R4/</code>
Admin-paneel	<code>/openmrs/admin/</code> — alleen intern bereikbaar?
Bestandsuploads	Modulebeheer: <code>.omod</code> -bestanden uploaden
Gebruikersinvoer	Zoekformulieren, vrije-tekstvelden
Omgevingsvariabelen	Database-wachtwoord, secret keys in <code>runtime.properties</code>
Dependencies	Elk Maven-pakket is een potentieel aanvalspad (SCA)

Uitgangspunt: als iets een *grens* overschrijdt (netwerk, processen, vertrouwensniveau), is het onderdeel van de attack surface.

Attack Surface in OpenMRS module?

- Welke ben je tegengekomen / denk je te gaan vinden?



Attack Surface Reduceren

Principe van least privilege + minimale blootstelling

Strategieën

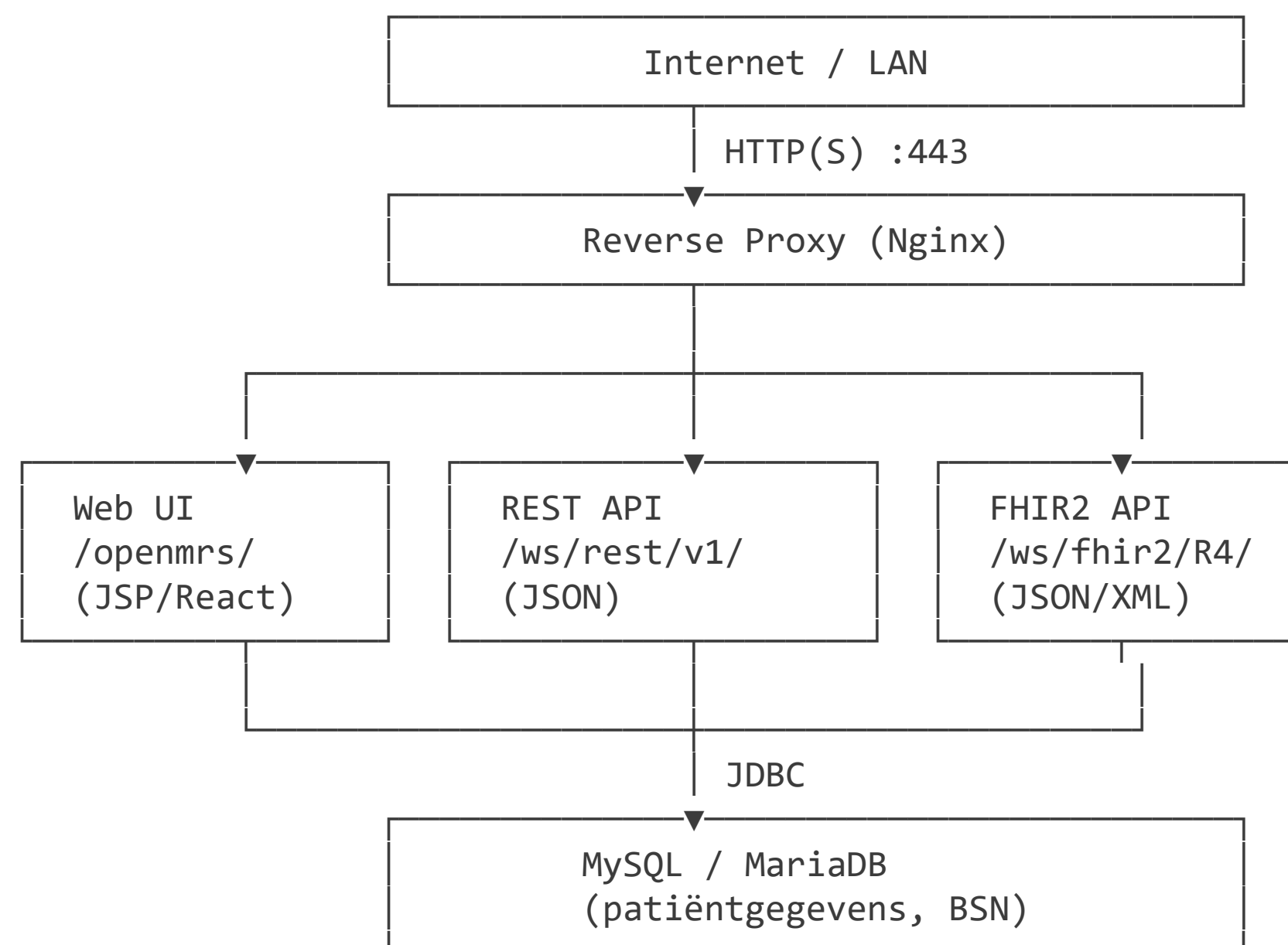
- Disable unused features
 - OpenMRS-modules die je niet gebruikt, verwijderen → minder code = minder aanvalspaden.
- Principle of Least Privilege (PoLP)
 - De databasegebruiker van de applicatie heeft alleen SELECT/INSERT/UPDATE — nooit DROP TABLE.
- Minimaliseer exposed endpoints
 - FHIR2-endpoint uitsluitend bereikbaar voor geauthenticeerde API-clients, niet publiek.
- Input validation aan de grens
 - Valideer altijd op de server, ook als je client-side validatie hebt.
- Verberg technische details
 - Stack traces nooit naar de browser sturen. HTTP-headers (b.v. *X-Powered-By, Server*) minimaliseren.

NEN-7510 8.25 vereist dat ongebruikte functionaliteit wordt uitgeschakeld of verwijderd.

Attack Surface

Hoe in beeld krijgen?

- Threat Modeling / Systeem overzicht



Aandachtspunten (rood = hoog risico):

- ● Admin-paneel /openmrs/admin/ — mag NIET publiek bereikbaar zijn
- ● Database-poort 3306 — mag NOOIT extern bereikbaar zijn
- ● REST API — vereist authenticatie én autorisatie per endpoint
- ● FHIR2 — vereist OAuth2/SMART-on-FHIR scope-validatie

OWASP Top 10 (2021)

#	Categorie	Kern van het probleem
A01	Broken Access Control	Gebruikers doen dingen die ze niet mogen
A02	Cryptographic Failures	Gevoelige data onversleuteld of slecht versleuteld
A03	Injection	Onvertrouwde input wordt als code uitgevoerd
A04	Insecure Design	Architecturale fouten, geen threat modeling
A05	Security Misconfiguration	Verkeerde of ontbrekende instellingen
A06	Vulnerable & Outdated Components	Niet-gepatchte dependencies (zie WS04)
A07	Identification & Authentication Failures	Zwakke login, geen MFA, slechte sessies
A08	Software & Data Integrity Failures	Onveilige CI/CD, unsigned artefacten
A09	Security Logging & Monitoring Failures	Aanvallen worden niet gedetecteerd
A10	Server-Side Request Forgery	Server wordt misbruikt om interne verzoeken te doen

NEN-7510 8.28 Veilig coderen vereist dat bekende kwetsbaarheidspatronen (zoals OWASP Top 10) worden vermeden.

OWASP A01 Broken Access Control

- **Probleem:** Een endpoint retourneert patiëntdata zonder te controleren of de ingelogde gebruiker bij *die* patiënt mag.

```
// ❌ FOUT: geen autorisatiecheck
@GetMapping("/patient/{uuid}")
public PatientDTO getPatient(@PathVariable String uuid) {
    return patientService.getPatientByUuid(uuid);
}

// ✅ GOED: Spring Security + OpenMRS privilege
@GetMapping("/patient/{uuid}")
@PreAuthorize("hasPrivilege('Get Patients')")
public PatientDTO getPatient(@PathVariable String uuid,
                             Authentication auth) {
    // Behandelrelatie-check: mag deze gebruiker deze patiënt zien?
    accessControlService.assertAccessToPatient(auth, uuid);
    return patientService.getPatientByUuid(uuid);
}
```

OWASP A02 Cryptographic Failures

- ● Nooit wachtwoorden plaintext opslaan → gebruik BCrypt (OpenMRS gebruikt dit al)
- ● Nooit hardcoded secrets in code: `String password = "Admin1234";`
- ✓ Gebruik `runtime.properties` of omgevingsvariabelen
- ✓ TLS verplicht voor alle verbindingen (extern én intern)

OWASP A03 Injection (SQL)

//  *FOUT: String concatenation → SQL-injectie mogelijk*
`String hql = "FROM Patient p WHERE p.identiflier = '" + input + "'";`

//  *GOED: Hibernate parameterized query*
`String hql = "FROM Patient p WHERE p.identiflier = :identiflier";
Query q = session.createQuery(hql);
q.setParameter("identiflier", input);`

klassieke voorbeeld: input ' ; DROP TABLE patients; --

OWASP A04 Insecure design

Probleem: Beveiligingsfouten die niet door technische controls op te lossen zijn (ze zitten in de architectuur).

- Geen threat modeling gedaan
 - risico's nooit geïdentificeerd
- Geen "secure defaults"
 - nieuw aangemaakte gebruiker krijgt automatisch admin-rechten
- Geen rate limiting op inlogpagina
 - brute-force mogelijk

Fix:

- Threat modeling / risico analyse verplicht in designfase
- Secure by default: nieuwe gebruiker = geen privileges totdat expliciet toegekend
- Rate limiting: spring-security-ratelimiter of via reverse proxy (Nginx limit_req)

OWASP A05 Security Misconfiguration

Paar puntjes mbt OpenMRS

Misconfiguratie	Risico	Fix
Debug-modus aan in productie	Stack traces zichtbaar voor aanvaller	<code>log.level=WARN</code> in productie
<code>allowedOrigins: *</code> in CORS-configuratie	Cross-origin API-aanroepen vanuit elke website	Whitelist specifieke origins
Default admin-wachtwoord niet gewijzigd	Triviale overname	Forceer wachtwoordwijziging bij eerste login
H2 console bereikbaar in productie	Directe databasetoegang	Schakel H2 console uit (<code>spring.h2.console.enabled=false</code>)
Tomcat met onnodige servlets	Grotere attack surface	Verwijder <code>DefaultServlet</code> , <code>JspServlet</code> indien ongebruikt

NEN-7510 8.28: Configuratie-instellingen die de beveiliging beïnvloeden, moeten worden gecontroleerd en beveiligd.

OWASP A06 Vuln. & Outdated Components

Wat is het risico?

- Software bevat doorgaans tientallen tot honderden externe libraries
- Eén library met een bekende CVE is voldoende voor een aanvaller
- In OpenMRS:
 - Spring Framework, Hibernate, Log4j, Jackson, FHIR2-bibliotheken, etc
- OWASP A06 'rising star'
 - Supply-chain-aanvallen nemen toe (SolarWinds, Log4Shell, XZ, Trivy (= SAST tool...))

Anders dan de overige Top 10?

- De kwetsbaarheid zit niet in 'eigen' code, maar in code van derden
- Geen 'code fix':
 - updaten, monitoren en scheiden
- Detectie vereist tooling:
 - SCA-scanner (Snyk, OWASP Dependency-Check) / SBOM

NEN-7510 8.8 eis: Kwetsbaarheden moeten tijdig worden geïdentificeerd én verholpen.

Documenteer de doorlooptijd (detectie → patch).

OWASP A07 Identification & auth failures

NEN-7510 8.5 – Beveiligde authenticatie

Risico's:

- Geen MFA voor admingebruikers
- Sessie-tokens worden niet geïnvalideerd na uitloggen
- Zwakke wachtwoordeisen
- Login-pogingen worden niet gelimiteerd (brute-force)

Fixes in OpenMRS/Spring:

```
// Sessie ongeldig maken bij uitloggen
@PostMapping("/logout")
public void logout(HttpSession session) {
    session.invalidate(); // ✅ Verplicht!
}

// Spring Security: account vergrendelen na 5 foute pogingen
http.formLogin()
    // ✅ Log de poging!
    .failureHandler(authenticationFailureHandler)
    .and()
    .sessionManagement()
    // ✅ Nieuw sessie-ID na login
    .sessionFixation().migrateSession();
```

OWASP A08 Software / Data Integrity Failure

Vorm	Beschrijving	OpenMRS-voorbeeld
Dependency Confusion	Aanvaller plaatst een kwaadaardig pakket met dezelfde naam op een publieke registry	Interne Maven-module openmrs-internal-auth wordt gevonden op Maven Central
CI/CD Pipeline Compromise	Build-artefact wordt gewijzigd ná het testen	.war-bestand wordt vervangen na de test-fase, vóór deploy
Unsigned artifacts	JAR-bestanden zonder checksums of handtekening	openmrs.war zonder SHA-256 verificatie

Mitigaties

- Gebruik pinned dependencies (<version> exact opgeven in pom.xml)
 - Ook GitHub Actions!
- Valideer checksums van gedownload artefacten: sha256sum openmrs.war
- Gebruik een private Maven proxy (Nexus/Artifactory) met expliciete allow-list
- Onderteken build-artefacten met GPG of Sigstore (<https://www.sigstore.dev/>)

NEN-7510 8.32 vereist bescherming van de softwareontwikkelomgeving

aanval niet gelogt betekend lastig te detecteren.

Wat loggen	Reden
Gefaalde inlogpogingen	Brute-force detectie
Succesvolle login + IP	Ongebruikelijke locatie detecteren
Toegang tot gevoelige endpoints	Ongeautoriseerde toegangspogingen
Wijzigingen in gebruikersrechten	Privilege escalation detecteren
Systeemfouten en exceptions	Incidentanalyse

OWASP A10 Server Side Req Forgery

Wat is het?

- Een aanvaller manipuleert een URL-parameter zodat de *server* een verzoek doet naar een intern systeem dat de aanvaller normaal niet kan bereiken.
- *"leent" de positie van de app server.*

NEN-7510 8.8 eis: Kwetsbaarheden moeten tijdig worden geïdentificeerd én verholpen.

Documenteer de doorlooptijd (detectie → patch).

OWASP A10 Server Side Req Forgery

Kwetsbaar

```
// KWETSBAAR - geen validatie van de URL
@PostMapping("/fetch-external-resource")
public ResponseEntity<String> fetchResource(@RequestParam String url) {
    RestTemplate restTemplate = new RestTemplate();
    String result = restTemplate.getForObject(url, String.class); // gevaarlijk!
    return ResponseEntity.ok(result);
}
```

Veilig

```
// VEILIG - allow-list van toegestane hosts
private static final List<String> ALLOWED_HOSTS = List.of(
    "fhir.example.nl",
    "api.openmrs.org"
);

@PostMapping("/fetch-external-resource")
public ResponseEntity<String> fetchResource(@RequestParam String url) {
    URI uri = URI.create(url);
    if (!ALLOWED_HOSTS.contains(uri.getHost())) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body("Host niet toegestaan: " + uri.getHost());
    }
    RestTemplate restTemplate = new RestTemplate();
    String result = restTemplate.getForObject(uri, String.class);
    return ResponseEntity.ok(result);
}
```

Voorbeeld Secure Coding checklist (Java / OpenMRS)

Input & Validatie

- [] Alle gebruikersinvoer gevalideerd server-side (niet alleen client-side)
- [] Parameterized queries gebruikt (geen string-concatenatie in SQL/HQL)
- [] Bestandsupload: type, grootte en inhoud gevalideerd
- [] XML-input: XXE-protectie ingeschakeld (`FEATURE_EXTERNAL_GENERAL_ENTITIES = false`)

Authenticatie & Autorisatie

- [] Elke endpoint heeft een expliciete `@PreAuthorize`-annotatie
- [] Sessie wordt geïnvallideerd na uitloggen
- [] Wachtwoorden gehasht met BCrypt (nooit MD5/SHA1)
- [] Geen hardcoded credentials in code of configuratie

Foutafhandeling & Logging

- [] Generieke foutmelding naar gebruiker; detail alleen in logbestand
- [] BSN, wachtwoord, sessie-tokens worden NOOIT gelogd
- [] Alle beveiligingsrelevante events worden gelogd (zie Slide 24)
- [] Exceptions worden niet geslokt (`catch (Exception e) {}`)

Configuratie

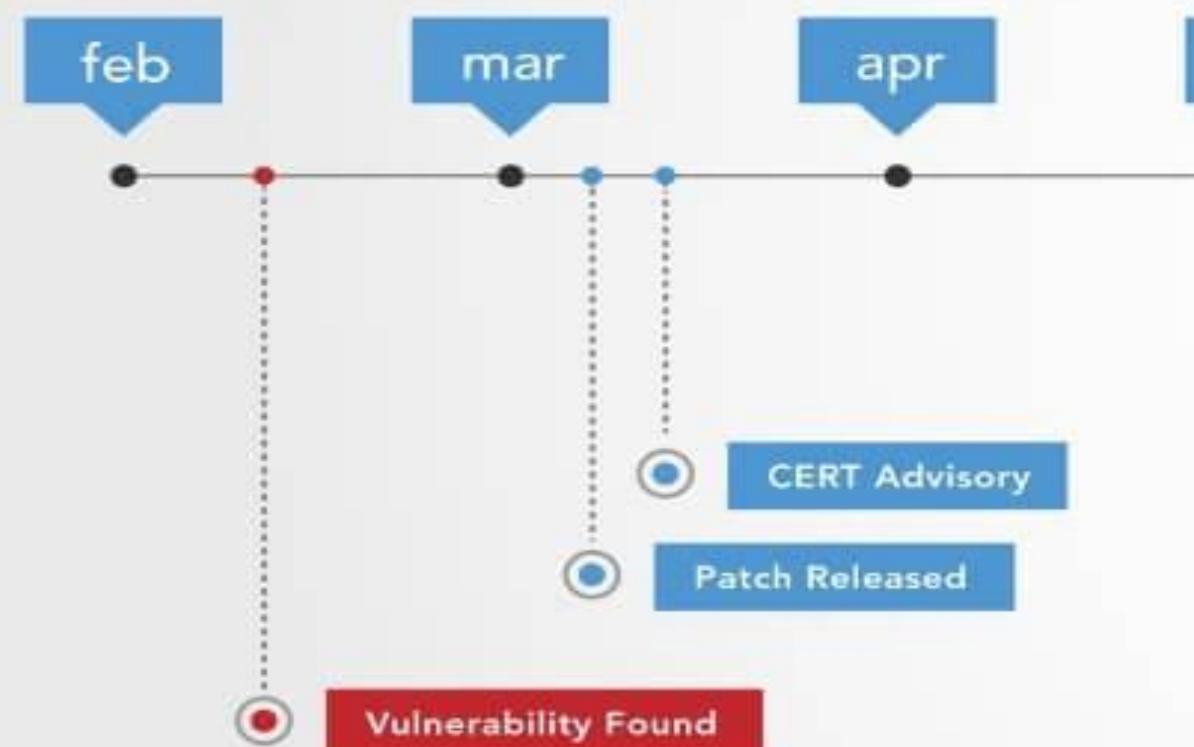
- [] Debug-modus uitgeschakeld in productie
- [] CORS-whitelist geconfigureerd (geen *)
- [] TLS verplicht (geen HTTP fallback)
- [] Dependencies up-to-date (Dependabot actief — zie WS02)

- **NEN-7510 8.28 Veilig coderen** vereist het hanteren van een gedocumenteerde secure-codingstandaard.

Wanneer het fout gaat (Equifax, 2017)

Equifax Data Breach Timeline

2017 details at csrps.com



CSR Data Breach Solutions from the Data Privacy

ars PRO++ AI BIZ & IT CARS CULTURE GAMING HEALTH POLICY SCIENCE SECURITY SPACE TECH FORUM GIFT

Critical vulnerability attack imperils [Updated]

Exploits for easy-to-spot bug are trivial,

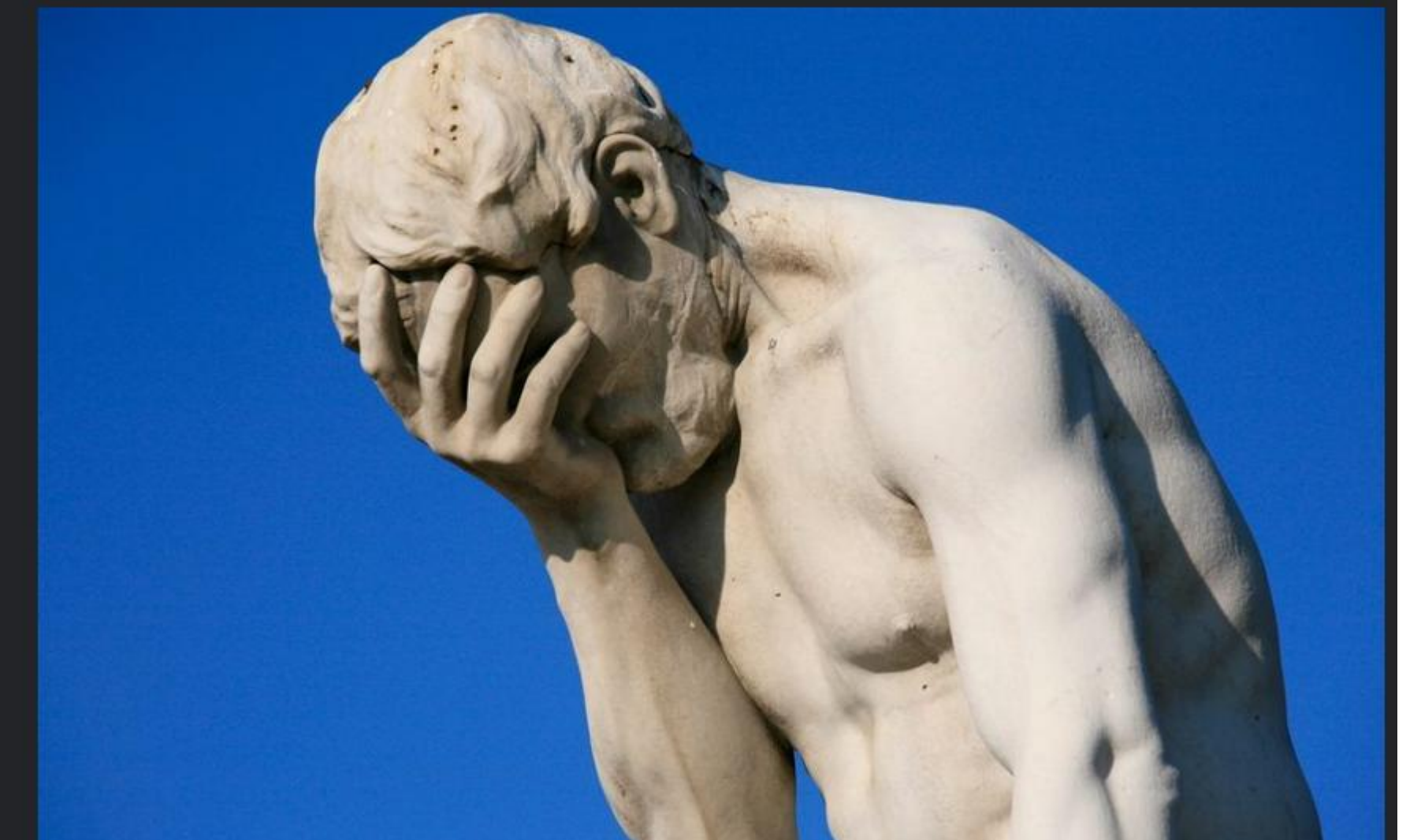
DAN GOODIN - MAR 9, 2017 7:07 PM | 65

ars PRO++ AI BIZ & IT CARS CULTURE GAMING HEALTH POLICY SCIENCE SECURITY SPACE TECH FORUM GIFT SUBS

Failure to patch two-month-old bug led to massive Equifax breach

Critical Apache Struts bug was fixed in March. In May, it bit ~143 million US consumers.

DAN GOODIN - SEP 14, 2017 5:12 AM | 237



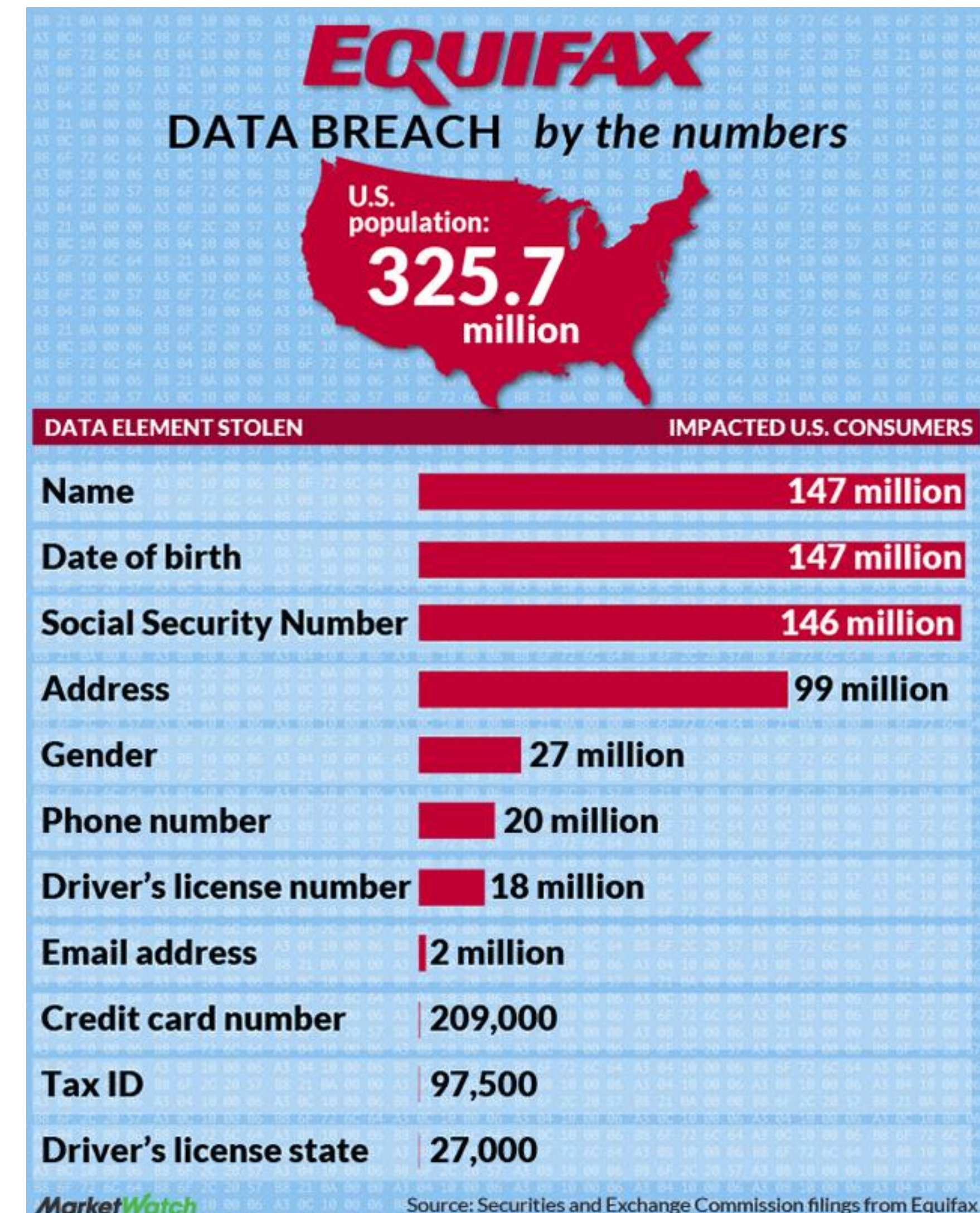
Apache Struts CVE-2017-5638

9 maart 2017!

Wanneer het fout gaat (Equifax, 2017)

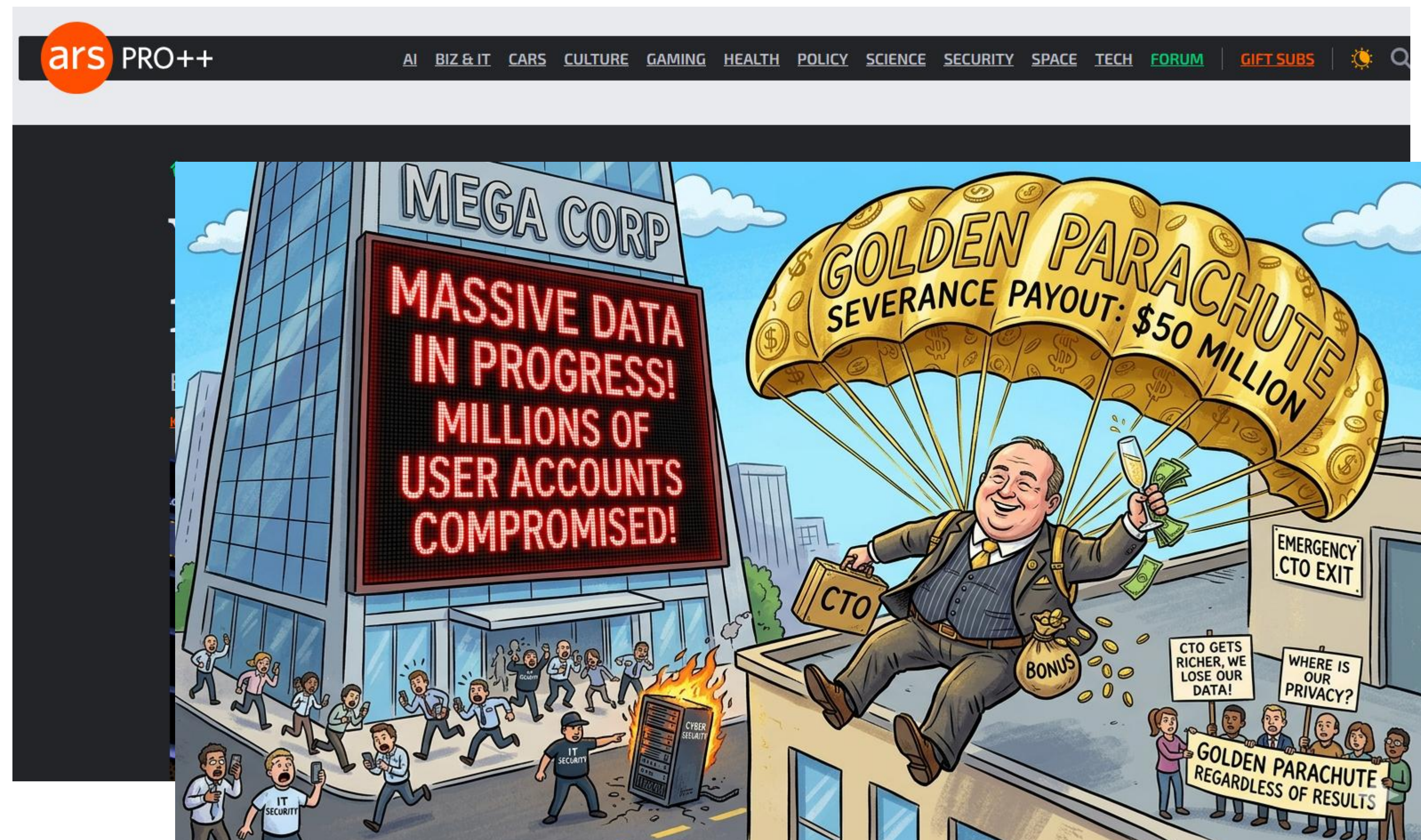
Datum	Gebeurtenis
7 maart 2017	Apache publiceert patch voor CVE-2017-5638 (CVSS: 10.0)
9 maart 2017	Equifax ontvangt interne melding: patch direct toepassen
13 mei 2017	Aanvaller begint aanval via niet-gepatchte Apache Struts server
29 juli 2017	Equifax ontdekt de inbraak (77 dagen later!)
7 sept 2017	Equifax maakt het datalek publiek
2019	Schikking van \$700 miljoen — CEO, CIO en CSO vertrekken

Apache Struts CVE-2017-5638



Wanneer het fout gaat (Equifax, 2017)

- Consequenties?
- Slachtoffers?
 - Directie (CEO, CIO)?



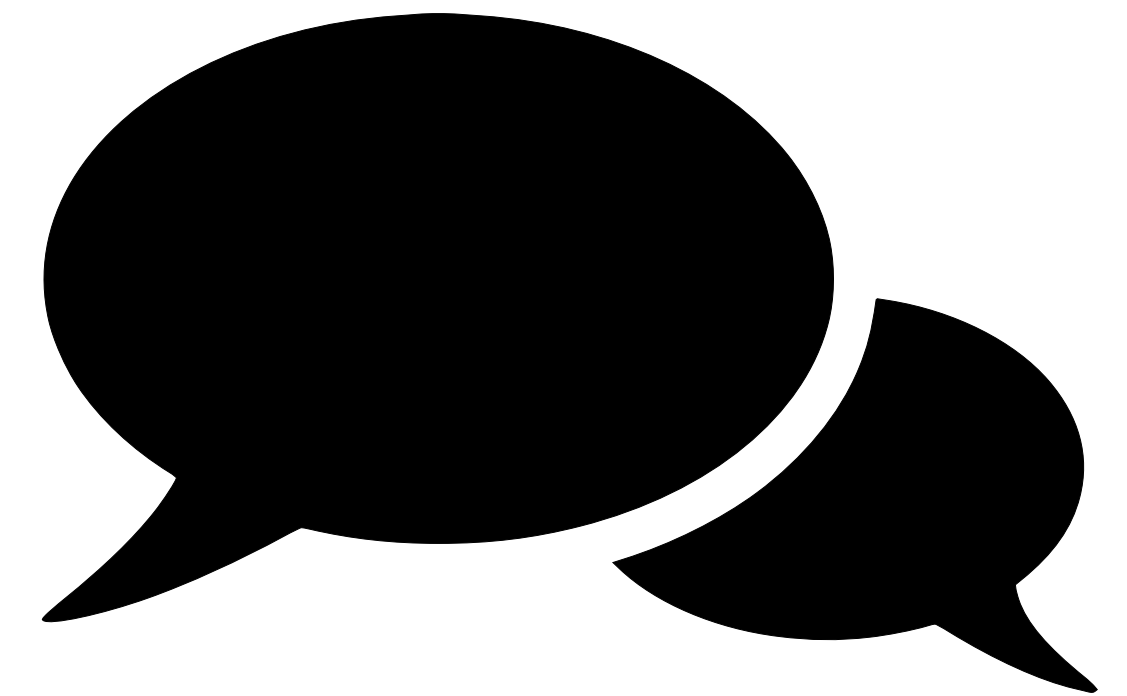
Apache Struts CVE-2017-5638

Equifax 2017 | Lessen

Fout	Wat had het kunnen voorkomen?	NEN-7510
Geen patch management proces	Patch binnen 24u voor CVSS ≥ 9	8.8 Beheer van technische kwetsbaarheden
Geen netwerksegmentatie	Aanvaller kon lateraal bewegen door alle systemen	8.22 Netwerksegregatie
Geen anomaliedetectie	77 dagen onopgemerkt (!)	8.16 Bewakingsactiviteiten
SSL-inspectie uitgeschakeld	Versleuteld verkeer niet gemonitord	8.9 Configuratiebeheer

OpenMRS 'lessen'

- Patch-deadline:
 - CVSS ≥ 9.0 → fix binnen 24 uur; CVSS 7–8.9 → binnen 7 dagen
- Segmenteer je netwerk:
 - Database alleen bereikbaar vanaf de applicatieserver
- Monitor actief:
 - Ingestelde alerts op ongebruikelijke query-volumes of API-aanroepen
- Test het patch-proces:
 - weet hoe snel een kritieke dependency geüpdate kan worden



NEN-7510 8.8, 8.9, 8.16, 8.22 zijn de vier maatregelen die de Equifax-aanval hadden kunnen stoppen of significant beperken.

*PRIVACY BY DESIGN
&
BY DEFAULT*

Privacy By Design | 7 principes (Cavoukian)

- Ann Cavoukian (2009); verankerd in de AVG (art. 25).

#	Principe	Betekenis voor OpenMRS
1	Proactief, niet reactief	Nadenken over privacyrisico's vóór de code geschreven wordt
2	Privacy als standaard	Zonder actie van de gebruiker maximale privacybescherming
3	Privacy ingebakken in het ontwerp	Niet als add-on, maar als kernfunctionaliteit
4	Volledige functionaliteit	Privacy én beveiliging zijn geen trade-off (both-and, not either-or)
5	End-to-end beveiliging	Bescherming gedurende de volledige levenscyclus van data
6	Zichtbaarheid en transparantie	Patiënt weet wat er met zijn data gebeurt
7	Respect voor gebruikersprivacy	Gebruiker heeft controle; data zijn zo min mogelijk verzameld

Privacy by Default & Dataminimalisatie

De sterkste privacyinstelling is de standaard. De gebruiker hoeft niets te doen om beschermd te zijn.

Voorbeelden in OpenMRS:

- Nieuw aangemaakt account: geen toegang tot patiëntgegevens totdat expliciet toegewezen
- API-responses bevatten standaard géén BSN (tenzij expliciet gevraagd met juiste privilege)
- Audit-logs worden standaard ingeschakeld (niet optioneel)

Dataminimalisatie (AVG art. 5 lid 1c)

"Persoonsgegevens moeten toereikend zijn, ter zake dienend en beperkt tot wat noodzakelijk is."

Vraag	Toepassing
Is dit gegeven noodzakelijk?	BSN opslaan is alleen verplicht als er een wettelijke grondslag is (bijv. declaratie)
Wat is de verwerkingsgrondslag?	AVG art. 6 (algemeen) of art. 9 (bijzondere categorieën, zoals gezondheidsdata)
Hoe lang bewaren we het?	Zo kort als mogelijk; medische gegevens: minimaal 20 jaar (WGBO art. 7:454)

Soort gegeven	Retentie
Patient record	20 jaar na laatste behandeling (wettelijk verplicht)
Audit logs	3–5 jaar (NEN-7510 8.15 + AVG)
Technische logs	90 dagen (operationeel; geen bijzondere persoonsgegevens)

Bijzondere Persoonsgegevens in de Zorg

Gezondheidsgegevens zijn bijzondere persoonsgegevens (AVG art. 9).

- **Zwaarder beschermingsregime** dan gewone persoonsgegevens.

Wat valt hieronder?

- Diagnoses, medicatie, laboratoriumuitslagen
- Psychologische of psychiatrische gegevens
- Genetische gegevens
- Biometrische gegevens (indien gebruikt voor identificatie)

Verwerking toegestaan als (AVG art. 9 lid 2):

- Expliciete toestemming van betrokkene (art. 9 lid 2a) — moeilijk in spoedgevallen
- Noodzakelijk voor gezondheidszorg door een zorgprofessional (art. 9 lid 2h) ← meest relevant
- Wettelijke verplichting (art. 9 lid 2b/c)

Privacy by default

- Standaard staan privacy instellingen op het strengste niveau
- Het is eenvoudig om naar deze 'default' terug te gaan
 - 'klik op de knop' principe
- Datamodel en autorisaties
 - Een arts kan alleen 'zijn' patienten zien, tenzij...
 - Een technisch support medewerker kan geen medische dossiers opvragen
 - Een patient kan alleen zijn eigen gegevens inzien

Privacy by default

- Denk ook aan:
 - Automatisch uitloggen of vergrendelen na inactiviteit
 - Tablets / printouts op een (open) afdeling
 - Maar: audit trail vereist voor compliancy ?
- Altijd bewuste afweging:
 - Gemak vs security & privacy
- NEN-7510:2024 deel 2
 - Control 8.11 vs control 8.16

DPIA: Data Protection Impact Assessment

- Verplicht bij grootschalige verwerking van bijzondere persoonsgegevens (AVG art. 35).
 - OpenMRS in ziekenhuis? Vrijwel altijd verplicht
- Sjabloon op Brightspace

Privacy by Design en de SDLC



SDLC-fase	Privacy-activiteit	NEN-7510 / AVG
Requirements	Privacy requirements formuleren; DPIA-check uitvoeren; verwerkingsgrondslag bepalen	AVG art. 25, 35; NEN-7510 5.34
Design	Datamodel beoordelen op dataminimalisatie; toegangsmodel op behandelrelatie-basis; retentiebeleid bepalen	NEN-7510 5.34, 8.11
Implementation	Pseudonimisering inbouwen; data masking in API-responses; geen bijzondere persoonsgegevens in logs	NEN-7510 8.11, 8.28
Testing	Privacy-tests: kan een ongeautoriseerde gebruiker BSN opvragen? Staan er persoonsgegevens in de logs?	NEN-7510 8.29
Release	DPIA herzien bij nieuwe functionaliteit; privacybeleid bijwerken	AVG art. 35
Operate	Toegangsverzoeken (art. 15) afhandelen; data-retentie handhaven; datalek melden (art. 33: binnen 72u)	AVG art. 15, 33; NEN-7510 5.34

- Behandelrelatie-gebaseerde toegang: In een EPD mag een arts alleen de patiënten zien die aan hem zijn toegewezen — niet alle patiënten in het systeem.

LOGGING

Vs Privacy?

Z-CERT: Nederlandse Healthcare CERT

- NEN-7510 8.15 zegt: log alles wat beveiligingsrelevant is.
- Privacy by Default zegt: verzamel zo min mogelijk persoonsgegevens.

Oplossing	Beschrijving
Pseudonimisering in logs	Sla in het logbestand een intern <code>patientId</code> op (UUID), niet het BSN of de naam. Mapping van UUID → BSN staat in een aparte, strikt beveiligde tabel.
Threshold-based alerting	Detecteer anomalieën op <i>gedragsniveau</i> (bijv. "gebruiker X opende >100 patiëntdossiers in 10 minuten") zonder de dossierinhoud te loggen.
Scheiding log-typen	Technische logs (voor ops): geanonimiseerd. Audit logs (voor compliance): pseudoniem + strikt toegangsbeheer.

NEN-7510 8.11 Data masking biedt de technische oplossing voor de juridische spanning.

Logging: Wat wel

- Log events, geen 'gegevens'

Event	Minimale log-inhoud
Inlogpoging (geslaagd)	Timestamp, User-ID, IP-adres, SUCCESS
Inlogpoging (mislukt)	Timestamp, gebruikersnaam (niet wachtwoord!), IP-adres, FAILURE
Toegang tot patiëntdossier	Timestamp, User-ID, Patient-UUID (niet BSN!), actie (READ)
Aanmaken/wijzigen patiëntrecord	Timestamp, User-ID, Patient-UUID, actie (CREATE/UPDATE)
Wijziging van gebruikersrechten	Timestamp, admin-User-ID, getroffen User-ID, nieuwe rol
Exporteren van data	Timestamp, User-ID, exporttype, aantal records
Systeemfouten	Timestamp, error-code, stack trace (geanonimiseerd)

Logging: Wat niet

Gegeven	Reden
Wachtwoorden (ook gehashed!)	Credential theft via logbestanden
BSN	Bijzonder persoonsgegeven; AVG art. 9
Diagnoses / medicatiegegevens	Bijzonder persoonsgegeven
Sessie-tokens / JWT-tokens	Token theft → session hijacking
Creditcardnummers	PCI-DSS verbod
Volledige namen in combinatie met medische data	Herleidbaar tot individu

NEN-7510 8.15: Logbestanden moeten gebruikersactiviteiten, uitzonderingen en informatiebeveiligingsgebeurtenissen vastleggen.

Log types / bestemmingen

Type	Doel	Doelgroep	Bewaartermijn	Inhoud
Technisch log	Debugging, performance, fouten opsporen	Developers, Ops	30–90 dagen	Stack traces, query-tijden, HTTP-statuscodes
Auditlog	Compliance, forensisch onderzoek, incident response	Auditors, CISO, Compliance	3–5 jaar	Wie deed wat wanneer op welke data (geen gevoelige inhoud)

NEN-7510 8.15: Logbestanden moeten gebruikersactiviteiten, uitzonderingen en informatiebeveiligingsgebeurtenissen vastleggen.

Eisen voor audit-logbestanden (NEN-751 8.15)

Elk audit-logbericht bevat minimaal:

- User-ID (wie voerde de actie uit?)
- Timestamp (inclusief tijdzone, gebruik ISO 8601)
- Event type (wat werd er gedaan?)
- Uitkomst (SUCCESS / FAILURE)
- Resource (op welk object werd gehandeld? (UUID, niet BSN))

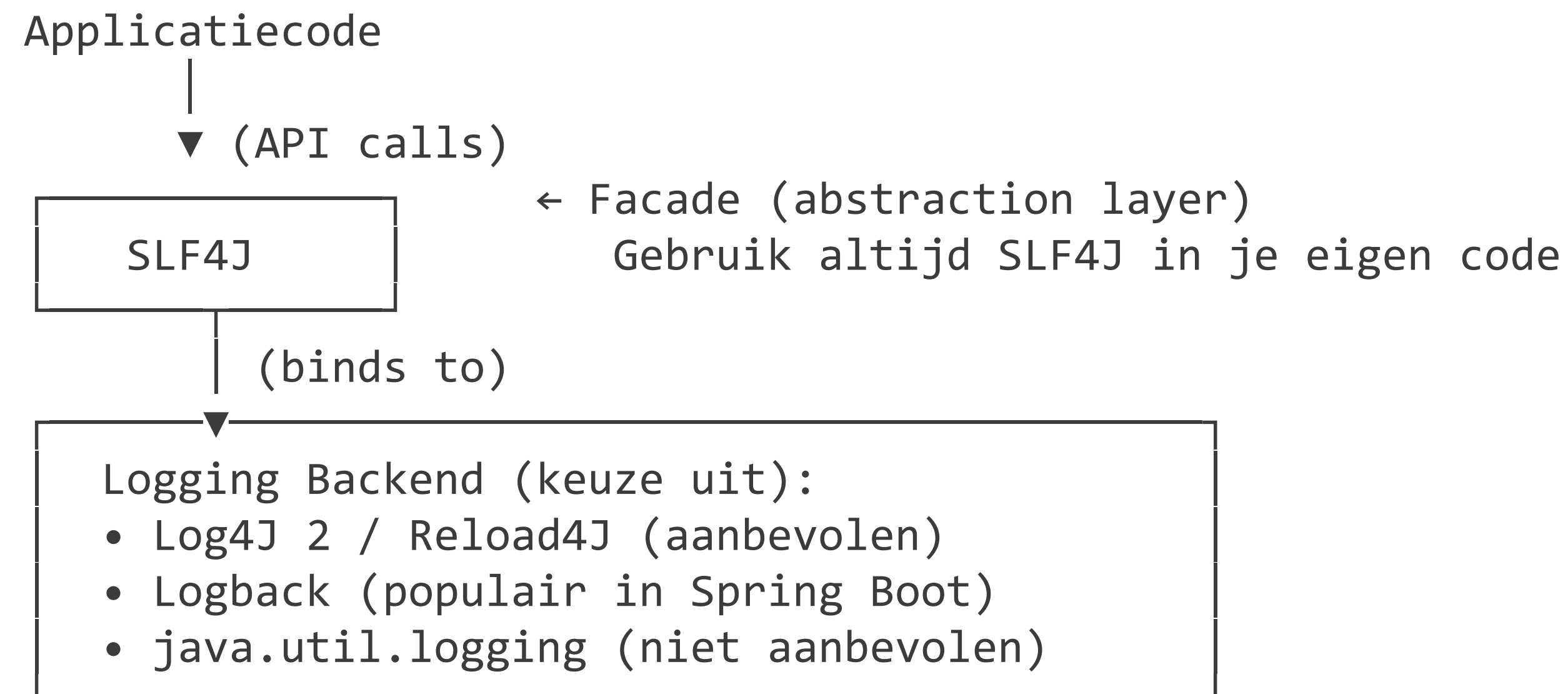
NEN-7510 8.15: Logbestanden moeten gebruikersactiviteiten, uitzonderingen en informatiebeveiligingsgebeurtenissen vastleggen.

LOGGING

Techniek

avans
hogeschool

Logging in Java: SLF4J + Log4J/Reload4J



NEN-7510:2024 (Maatregel 8.15: Logregistratie)

Logs | bestemming

- Flat files
- Database
- Service
 - Journald (onderdeel van systemd), (r)syslog, Events
- Logserver (on prem / SAAS)
- Gebruik een logging framework dat uitbreidbaar en configureerbaar is
 - (zonder codewijzigingen)
 - Log4J, java.util.logging, Reload4J
 - Ook: SLF4J
 - Facade, maakt binding met een concrete logger, b.v. Log4J

Log framework relevant voor security?

CVE-2021-44228 – Log4Shell (CVSS: 10.0)

- Datum: December 2021
- Getroffen: Log4J 2.x
- Impact: Remote Code Execution op elke server die Log4J gebruikt
- **Note:** Log4J was default op elke server die Log4J gebruikt



Les: Neem ALLES mee bij bepalen attack surface / threat modelling

NEN-7510 8.8 Beheer van technische kwetsbaarheden + 8.28 Veilig

coderen: framework-keuze is een beveiligingsbeslissing.

Logging tests?

Een ontbrekend logbericht is een **bug met compliance-gevolgen**.

Als een auditlog-entry ontbreekt, is er een gap in het audittrail — dit is een bevinding bij een NEN-7510-audit.

- Gebruik b.v.
 - Mockito verify
 - JaCoCo (ListAppender pattern)

Mockito voorbeeld

```
@ExtendWith(MockitoExtension.class)
class PatientAuditServiceTest {

    @Mock
    private Logger auditLog; // Gemockte Logger

    @InjectMocks
    private PatientAuditService auditService;

    @Test
    void shouldLogPatientAccessOnSuccess() {
        // Act
        auditService.logPatientAccess("patient-uuid-123", "READ", true);

        // Assert: verifieer dat de Logger werd aangeroepen
        verify(auditLog).info(
            contains("event=PATIENT_ACCESS"),
            anyString(), // userId
            anyString(), // timestamp
            eq("READ"), // action
            eq("patient-uuid-123"), // patientUuid
            eq("SUCCESS") // outcome
        );
    }

    @Test
    void shouldLogPatientAccessOnFailure() {
        auditService.logPatientAccess("patient-uuid-123", "READ", false);
        verify(auditLog).info(contains("outcome=FAILURE"),
            any(), any(), any(), any(), any());
    }
}
```

En dan dit nog

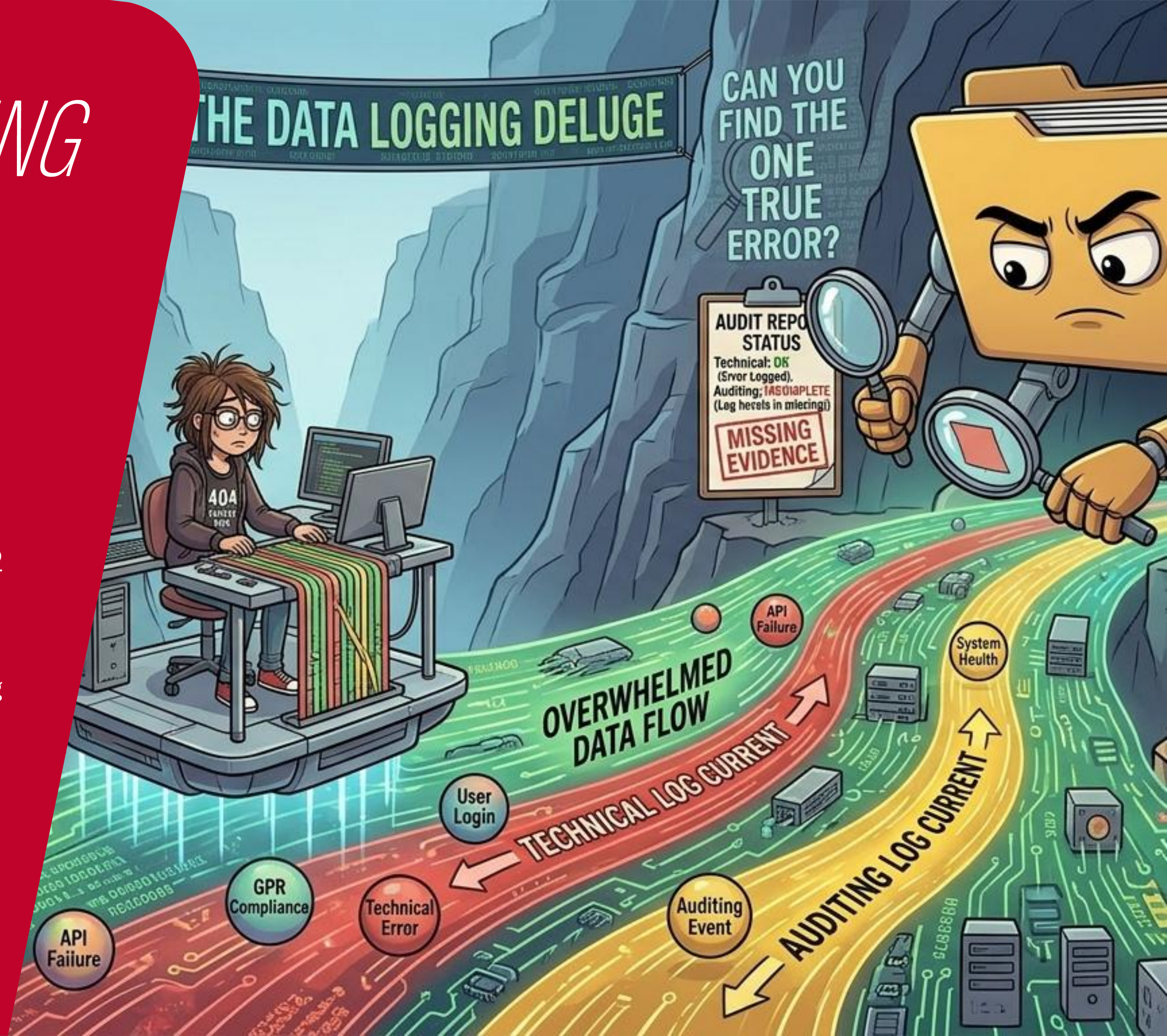
- Code coverage
 - Jacoco ondersteund dit
- Test rapportage
- **NEN-7510 8.29:** *Beveiligingstesten moeten aantonen dat beveiligingsfuncties correct werken.* Een testrapport met 0% coverage op de audit-service is een auditbevinding.

OPDRACHT LOGGING

Gebruik de gekozen OpenMRS module

Pre condities

- OpenMRS module gekozen
- Project (github) is ingericht
 - Conform eisen NEN-7510:2024-2
 - CI-CD is ingericht
 - Inc SAST, SBOM en SCA tooling



Opdracht | 5 onderdelen

Onderdeel	Beschrijving	Deliverable	NEN-7510
Deel 1	Attack surface mapping	Diagram + tabel van alle entry points	8.25
Deel 2	Logging gap-analyse	Document: huidige vs. vereiste logging	8.15
Deel 3	Logging implementeren	Java-code: compliant audit logging	8.28
Deel 4	Unit tests schrijven	JUnit + Mockito of ListAppender tests	8.29
Deel 5	JaCoCo in CI-pipeline	Coverage-rapport als GitHub Actions artifact	8.29

1 | Attack Surface Mapping

Doel: Breng alle entry points van jouw OpenMRS-module in kaart.

Stappen

- Identificeer alle HTTP-endpoints in jouw module
 - Zoek op @GetMapping, @PostMapping, @RequestMapping in de broncode
 - Noteer het pad, het HTTP-methode en de vereiste privileges
- Documenteer externe inputs
 - Welke velden accepteren gebruikersinvoer?
 - Zijn er bestandsuploads?
 - Zijn er omgevingsvariabelen die worden ingelezen?
- Maak een diagram (Thread model)
 - Teken het vertrouwensmodel: welke componenten vertrouwen elkaar?
 - Markeer hoge-risico entry points (rood/geel)
- Koppel aan NEN-7510 8.25
 - Per entry point: is er inputvalidatie? Is er autorisatiecheck?
 - Documenteer de gaps
- Deliverable
 - Thread model & attack surface overzicht. Kolommen:
 - Endpoint | Methode | Vereiste privilege | Inputvalidatie aanwezig? | Autorisatiecheck aanwezig?

2 | Logging gap analyse

Doel: Inzichtelijk krijgen wat er mist qua logging.

- Zoek alle bestaande log-statements in jouw module
 - `console.log('...')`
 - Of GenAI tool
- Maak een tabel:

Log statement	Log level	Log message	Log timestamp
<code>console.log('...')</code>	...	...	...
- Identificeer welke beveiligingsrelevante events **niet** worden gelogd
- Controleer of er gevoelige data (b.v. BSN, wachtwoorden) in de logs staat
- Documenteer de gap

3 | Logging implementeren

Doel: Logging compleet maken

- (optioneel) Maak een AuditLogService-klasse
- Voeg audit-logging toe aan minimaal:
- Inloggen / uitloggen
- Lezen van een patiëntdossier
- Aanmaken/wijzigen van een record
- Zorg dat elk logbericht voldoet aan NEN-7510 8.15
 - (TröðHĐپ şhlörşglپپ ðūðlşپپ tħşjnłrşپپ ððrņtðæðçŦŦHD)
 - UserID: (Context.getAuthenticatedUser())
- Zorg dat er geen BSN of medische data in de logs staat

4 | Logging testen

Doel: Logging compleet maken

- Schrijf een aantal JUnit-tests voor de logging
- Test dat een succesvolle toegang wordt gelogd
- Test dat een mislukte toegang wordt gelogd
- Test dat het logbericht geen BSN bevat (negatieve test)
- Gebruik ListAppender of Mockito (keuze)
- Alle tests slagen → mvn test geeft groen

5 | Code Coverage

Doel: Logging compleet maken

- Configureer & activeer code coverage, b.v. Jacoco:
 - mvn jacoco:report
 - (resultaat onder `omod/target/site/jacoco/index.html`)
- Bepaal een realistisch coverage %
 - Documenteer: waarom niet hoger & waarom niet lager
- Coverage rapport als artefact in GitHub Action

Gereed voor volgende workshop

Deliverable	Gebruik als Formeel Auditrapport (WS 6)
Attack surface diagram	Bewijsmateriaal: scope van de audit
Logging gap-analyse	Bevinding + aanbeveling in het rapport
Audit logging code (PR/commit)	Bewijs van remediation (maatregel getroffen)
Unit tests + JaCoCo rapport	Bewijs dat de maatregel werkt en getest is
Optioneel: DAST-rapport (ZAP)	Dynamisch testbewijs naast SAST (CodeQL, Snyk)

Rode draad

WS01: Regelgeving & begrippen



WS02: Pipeline beveiligen (SAST, SBOM, branch protection)



WS03: Risico's identificeren (threat modeling, risicomatrix)



WS04: Kwetsbaarheden scannen (CVE, Snyk, security backlog)



WS05: Kwetsbaarheden fixen + Privacy inbouwen ← HIER ZIJN WE NU

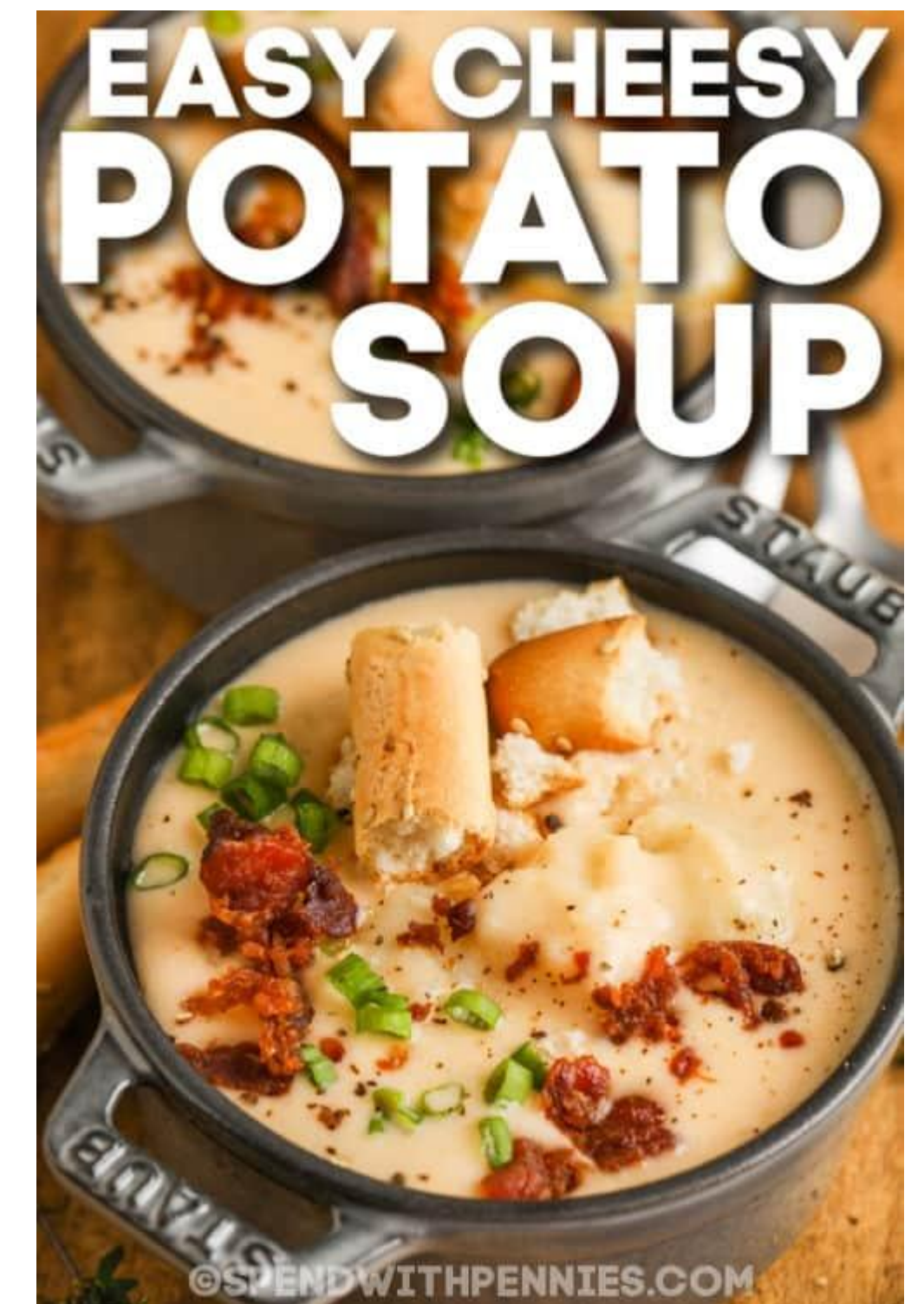


WS06: Formeel auditrapport (alle bewijsstukken samenbrengen)

Behandelde NEN-7510:2024-2 controls

Control	Omschrijving
8.25	Beveiligen tijdens de ontwikkelcyclus
8.28	Veilig coderen
8.29	Beveiligingstesten
8.15	Logregistratie
8.11	Data masking
5.34	Privacy en bescherming van persoonsgegevens
8.5	Beveiligde authenticatie
8.8	Beheer van technische kwetsbaarheden

- QOTD:
 - Beveiliging en privacy zijn geen obstakels voor goede software; het zijn kenmerken van goede software.
 - Een EPD zonder audit-logging is geen EPD; het is een aansprakelijkheidsrisico.





TOT SLOT.

Vragen?

Voorbeeld logregel

```
private static final Log log = LoggerFactory.getLog(MyService.class);

public Patient savePatientSecretData(Patient patient) {
    try {
        // Voer de actie uit
        Patient saved = dao.save(patient);

        // NEN-7510 Audit Log: Succesvolle wijziging
        log.info(String.format("AUDIT: User [%s] has updated patient data",
            Context.getAuthenticatedUser().getUsername(),
            patient.getUuid(),
            new Date()));

        return saved;
    } catch (Exception e) {
        // Technische Log: Foutmelding (geen patiëntdata in log)
        log.error("Failed to save patient data for patient %s", patient.getUuid(), e);
        throw e;
    }
}
```

Voorbeeld logregel test

- Een logregel is code, en code moet getest worden. In JUnit kan een ListAppender of een Mockito verify gebruikt worden om te controleren of de logger is aangeroepen.
- <https://openmrs.atlassian.net/wiki/spaces/Archives/pages/25468257/Module+Unit+Testing>

```
@Test
public void shouldLogAuditMessageWhenPatientIsSaved() {
    // 1. Setup
    Patient patient = new Patient();
    patient.setUuid("test-uuid");

    // 2. Execute
    myService.savePatientSecretData(patient);

    // 3. Verify (Conceptueel: controleer of de log.info is aangeroepen)
    // Tip: In OpenMRS kun je de 'BaseModuleContextSensitiveTest' gebruiken
    // om de geauthenticeerde gebruiker te mocken.
}
```

- [illegible]